

PRACTICAL EXAMPLES(WEEK-2)

Name-K.Vaishali
RollNo-2520030117
Section-7
subject-DBS&DBE

STEP-1 CREATE DATABASE

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Server Enterprise' tree with 'Databases' selected. The right pane shows the 'SQL Server Enterprise' window with the following SQL script:

```
1 CREATE DATABASE IF NOT EXISTS joins_db;
2 USE joins_db;
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
1	21:00:47	CREATE DATABASE IF NOT EXISTS joins_db	1 row(s) affected, 1 warning(s): 1007 Can't create database 'joins_db'; database exists
2	21:00:47	USE joins_db	0 row(s) affected

STEP-2 CREATE ORDER TABLE

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Server Enterprise' tree with 'Databases' selected. The right pane shows the 'SQL Server Enterprise' window with the following SQL script:

```
1 CREATE DATABASE IF NOT EXISTS orders_db;
2 USE orders_db;
3 CREATE TABLE orders (
4     ord_no INT PRIMARY KEY,
5     purch_amt DECIMAL(10,2),
6     ord_date DATE,
7     customer_id INT,
8     salesman_id INT
9 );
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
1	21:00:47	CREATE DATABASE IF NOT EXISTS joins_db	1 row(s) affected, 1 warning(s): 1007 Can't create database 'joins_db'; database exists
2	21:00:47	USE joins_db	0 row(s) affected
3	21:02:02	CREATE DATABASE IF NOT EXISTS orders_db	1 row(s) affected
4	21:02:02	USE orders_db	0 row(s) affected
5	21:02:02	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cus...	0 row(s) affected

STEP-3 INSERT DATA INTO ORDERS

The screenshot shows the SQL Developer interface with the 'orders' table created and 12 rows of data inserted. The table structure is as follows:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.50	2012-08-17	3009	5003
70007	948.50	2012-09-10	3005	5002
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.40	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.60	2012-04-25	3002	5001

The output window shows the following actions:

- 1 21:00:47 CREATE DATABASE IF NOT EXISTS joins_db (1 row(s) affected, 1 warning(s): 1007 Can't create database 'joins_db'; database exists)
- 2 21:00:47 USE joins_db (0 row(s) affected)
- 3 21:02:02 CREATE DATABASE IF NOT EXISTS orders_db (1 row(s) affected)
- 4 21:02:02 USE orders_db (0 row(s) affected)
- 5 21:02:02 CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cus... (0 row(s) affected)
- 6 21:03:30 INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,5005), ... (12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0)

STEP-4 WHERE CLAUSE

The screenshot shows the SQL Developer interface with the 'orders' table and the execution of a SELECT query. The query is:

```
SELECT * FROM orders WHERE purch_amt > 2000;
```

The result grid shows the following data:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

The output window shows the following actions:

- 10 21:05:57 CREATE DATABASE IF NOT EXISTS orders_db (1 row(s) affected, 1 warning(s): 1007 Can't create database 'orders_db'; database exists)
- 11 21:05:57 USE orders_db (0 row(s) affected)
- 12 21:05:57 DROP TABLE IF EXISTS orders (0 row(s) affected)
- 13 21:05:57 CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ... (0 row(s) affected)
- 14 21:05:57 INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,5005), ... (12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0)
- 15 21:05:57 SELECT * FROM orders WHERE purch_amt > 2000 (4 row(s) returned)

STEP-5 WHERE CLAUSE

K.Vaishali Local instancePR...

Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2

SQLAdditions: ...

Automatic context help is disabled. Use the caret position or to t

```

23 (70012,250.45,'2012-06-27',3008,5002),
24 (70011,75.29,'2012-08-17',3003,5007),
25 (70013,3045.60,'2012-04-25',3002,5001);
26 SELECT *
27 FROM orders
28 WHERE purch_amt > 2000;
29 SELECT *
30 FROM orders
31 WHERE ord_date='2012-09-10';

```

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005

orders 2

Output

#	Time	Action	Message
11	21:05:57	USE orders_db	0 row(s) affected
12	21:05:57	DROP TABLE IF EXISTS orders	0 row(s) affected
13	21:05:57	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ...	0 row(s) affected
14	21:05:57	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
15	21:05:57	SELECT * FROM orders WHERE purch_amt > 2000	4 row(s) returned
16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned

STEP-6 WHERE CLAUSE

K.Vaishali Local instancePR...

Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2

SQLAdditions: ...

Automatic context help is disabled. Use the caret position or to t

```

26 SELECT *
27 FROM orders
28 WHERE purch_amt > 2000;
29 SELECT *
30 FROM orders
31 WHERE ord_date='2012-09-10';
32 SELECT *
33 FROM orders
34 WHERE salesman_id=5001;

```

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

orders 3

Output

#	Time	Action	Message
12	21:05:57	DROP TABLE IF EXISTS orders	0 row(s) affected
13	21:05:57	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ...	0 row(s) affected
14	21:05:57	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
15	21:05:57	SELECT * FROM orders WHERE purch_amt > 2000	4 row(s) returned
16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned
17	21:08:33	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned

STEP-7 ORDER BY

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2

SQLAdditions: Automatic context help is disabled. Use the caret position or to to

```
29 • SELECT *
30 FROM orders
31 WHERE ord_date='2012-09-10';
32 • SELECT *
33 FROM orders
34 WHERE salesman_id=5001;
35 • SELECT *
36 FROM orders
37 ORDER BY purch_amt DESC;
```

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
700170010	3.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003

orders 4

Output

Action Output

#	Time	Action	Message
✓ 13	21:05:57	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ...	0 row(s) affected
✓ 14	21:05:57	INSERT INTO orders VALUES (70001,150.50;2012-10-05;3005,5002), (70009,270.65;2012-09-10;3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 15	21:05:57	SELECT * FROM orders WHERE purch_amt > 2000	4 row(s) returned
✓ 16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned
✓ 17	21:08:33	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned
✓ 18	21:14:36	SELECT * FROM orders ORDER BY purch_amt DESC	12 row(s) returned

STEP-8 ORDER BY

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2

SQLAdditions: Automatic context help is disabled. Use the caret position or to to

```
32 • SELECT *
33 FROM orders
34 WHERE salesman_id=5001;
35 • SELECT *
36 FROM orders
37 ORDER BY purch_amt DESC;
38 • SELECT *
39 FROM orders
40 ORDER BY ord_date;
```

Result Grid

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001

orders 5

Output

Action Output

#	Time	Action	Message
✓ 14	21:05:57	INSERT INTO orders VALUES (70001,150.50;2012-10-05;3005,5002), (70009,270.65;2012-09-10;3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 15	21:05:57	SELECT * FROM orders WHERE purch_amt > 2000	4 row(s) returned
✓ 16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned
✓ 17	21:08:33	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned
✓ 18	21:14:36	SELECT * FROM orders ORDER BY purch_amt DESC	12 row(s) returned
✓ 19	21:15:34	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned

STEP-9 SUM()

The screenshot shows the SQL Enterprise Manager interface for a local instance. The query editor displays the following SQL code:

```
34 WHERE salesman_id=5001;
35 • SELECT *
36 FROM orders
37 ORDER BY purch_amt DESC;
38 • SELECT *
39 FROM orders
40 ORDER BY ord_date;
41 • SELECT SUM(purch_amt) AS total_revenue
42 FROM orders;
```

The Results pane shows a single row with the value 17541.18 for the column total_revenue.

The Output pane shows the execution log:

#	Time	Action	Message
15	21:05:57	SELECT * FROM orders WHERE purch_amt > 2000	4 row(s) returned
16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned
17	21:08:33	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned
18	21:14:36	SELECT * FROM orders ORDER BY purch_amt DESC	12 row(s) returned
19	21:15:34	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned
20	21:16:24	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned

STEP-10 AVG()

The screenshot shows the SQL Enterprise Manager interface for a local instance. The query editor displays the following SQL code:

```
36 FROM orders
37 ORDER BY purch_amt DESC;
38 • SELECT *
39 FROM orders
40 ORDER BY ord_date;
41 • SELECT SUM(purch_amt) AS total_revenue
42 FROM orders;
43 • SELECT AVG(purch_amt) AS average_order
44 FROM orders;
```

The Results pane shows a single row with the value 1461.765000 for the column average_order.

The Output pane shows the execution log:

#	Time	Action	Message
16	21:07:32	SELECT * FROM orders WHERE ord_date='2012-09-10'	3 row(s) returned
17	21:08:33	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned
18	21:14:36	SELECT * FROM orders ORDER BY purch_amt DESC	12 row(s) returned
19	21:15:34	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned
20	21:16:24	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned
21	21:17:11	SELECT AVG(purch_amt) AS average_order FROM orders	1 row(s) returned

STEP-11 MAX()

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2 x SQLAdditions

Don't Limit

```

36 FROM orders
37 ORDER BY purch_amt DESC;
38 • SELECT *
39 FROM orders
40 ORDER BY ord_date;
41 • SELECT SUM(purch_amt) AS total_revenue
42 FROM orders;
43 • SELECT AVG(purch_amt) AS average_order
44 FROM orders;

```

Automatic context t

Result Grid

highest_order
5760.00

Form Editor

Field Types

orders 10 orders 11 orders 12 Result 13 Result 14 Read Only x Context Help Snippets

Output

Action Output

#	Time	Action	Message
29	21:17:58	SELECT * FROM orders WHERE salesman_id=5001	4 row(s) returned
30	21:17:58	SELECT * FROM orders ORDER BY purch_amt DESC	12 row(s) returned
31	21:17:58	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned
32	21:17:58	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned
33	21:17:58	SELECT AVG(purch_amt) AS average_order FROM orders	1 row(s) returned
34	21:17:58	SELECT MAX(purch_amt) AS highest_order FROM orders	1 row(s) returned

STEP-12 MIN()

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2 x Administration - Data Import/Res...

Don't Limit

```

40 ORDER BY ord_date;
41 • SELECT SUM(purch_amt) AS total_revenue
42 FROM orders;
43 • SELECT AVG(purch_amt) AS average_order
44 FROM orders;
45 • SELECT MIN(purch_amt) AS lowest_order
46 FROM orders;
47 • SELECT MAX(purch_amt) AS highest_order
48 FROM orders;

```

Automatic context t

Result Grid

highest_order
5760.00

Form Editor

Field Types

Result 16 Result 17 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
31	21:17:58	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned
32	21:17:58	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned
33	21:17:58	SELECT AVG(purch_amt) AS average_order FROM orders	1 row(s) returned
34	21:17:58	SELECT MAX(purch_amt) AS highest_order FROM orders	1 row(s) returned
35	21:19:40	SELECT MIN(purch_amt) AS lowest_order FROM orders	1 row(s) returned
36	21:19:40	SELECT MAX(purch_amt) AS highest_order FROM orders	1 row(s) returned

STEP-13 COUNT()

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2 Administration - Data Import/Res...

SQL Additions

Automatic context help

```
42 FROM orders;
43 SELECT AVG(purch_amt) AS average_order
44 FROM orders;
45 SELECT MIN(purch_amt) AS lowest_order
46 FROM orders;
47 SELECT MAX(purch_amt) AS highest_order
48 FROM orders;
49 SELECT COUNT(*) AS total_orders
50 FROM orders;
```

Result Grid

total_orders
12

Filter Rows: Export: Wrap Cell Contents: Result Grid Form Editor Field Types

orders 22 Result 23 Result 24 Result 25 Result 26 Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
46	21:20:46	SELECT * FROM orders ORDER BY ord_date	12 row(s) returned
47	21:20:46	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned
48	21:20:46	SELECT AVG(purch_amt) AS average_order FROM orders	1 row(s) returned
49	21:20:46	SELECT MIN(purch_amt) AS lowest_order FROM orders	1 row(s) returned
50	21:20:46	SELECT MAX(purch_amt) AS highest_order FROM orders	1 row(s) returned
51	21:20:46	SELECT COUNT(*) AS total_orders FROM orders	1 row(s) returned

Info Session

STEP-14 GROUP BY()

K.Vaishali Local instancePR...

File Edit View Query Database Server Tools Scripting Help

PracticalExampleweek-2 Administration - Data Import/Res...

SQL Additions

Automatic context help

```
46 FROM orders;
47 SELECT MAX(purch_amt) AS highest_order
48 FROM orders;
49 SELECT COUNT(*) AS total_orders
50 FROM orders;
51 SELECT salesman_id,
52 SUM(purch_amt) AS total_sales
53 FROM orders
54 GROUP BY salesman_id;
```

Result Grid

salesman_id	total_sales
5002	1349.45
5001	11271.46
5003	2590.90
5005	270.65
5006	1983.43
5007	75.29

Filter Rows: Export: Wrap Cell Contents: Result Grid Form Editor Field Types

Result 28 Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
47	21:20:46	SELECT SUM(purch_amt) AS total_revenue FROM orders	1 row(s) returned
48	21:20:46	SELECT AVG(purch_amt) AS average_order FROM orders	1 row(s) returned
49	21:20:46	SELECT MIN(purch_amt) AS lowest_order FROM orders	1 row(s) returned
50	21:20:46	SELECT MAX(purch_amt) AS highest_order FROM orders	1 row(s) returned
51	21:20:46	SELECT COUNT(*) AS total_orders FROM orders	1 row(s) returned
52	21:22:03	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned

Info Session

